Roll Back the Fault, Not the Future: Causal Recovery for Coding Agents with AGENTTX

Anonymous submission

Abstract

Coding agents increasingly execute long, stateful trajectories in which later tool calls reuse filesystem state produced by earlier editing, compilation, and testing. Existing mechanisms couple the lifetime of speculative state to the granularity of recovery: command-scoped isolation repeatedly reconstructs state, whereas coarse temporal rollback discards independent later work. Recovery may target a historical state that never existed as a temporal prefix, and the workspace and agent context must move to that target together. AGENTTX keeps the trajectory in one speculative workspace but recovers only the failed step and its dependents. It captures dependencies and object versions, then uses one recovery decision to rebuild both the workspace and agent-visible context. In the 144-run controlled recovery suite, AGENTTX retained 100% of independent work and removed 100% of invalid descendants; temporal recovery retained only 41% of independent work at 64 calls. At 48 document entries, preserving valid later work avoided 1,425 replay tokens relative to temporal recovery and 3,340 relative to whole-session abort.

1 Introduction

Coding agents increasingly solve software-engineering tasks through long sequences of interactions with a development environment rather than producing a patch in a single turn. Their control loops interleave model reasoning with tool actions and observations, allowing plans to change after commands fail or reveal new repository evidence [1]. Systems such as SWE-agent and OpenHands expose repository navigation, editing, shell commands, and tests to the model [2,3]. SWE-bench evaluates issue resolution across multiple files and test runs [4]. The filesystem therefore becomes persistent execution state. Later steps consume files and artifacts produced earlier in the same trajectory.

Existing systems preserve this evolving state at different boundaries. Command-level semisolates such as `try` stage one opaque invocation [5], whereas agent-facing filesystems and branch runtimes preserve longer-lived workspace state [6,7]. Checkpoints restore temporal state [8,9]; process checkpointing provides restart guarantees [10, 11]. These mechanisms usually recover the state unit they preserve: a command, session, branch, checkpoint, or transaction. Short units repeatedly reconstruct execution state; coarse units can remove valid work together with corrupted effects.

Correct recovery exposes this limitation. Leaving all state in place preserves progress, while removing only the failed operation can leave derived effects behind. Coarse recovery removes those effects, yet also discards unrelated work enclosed by its boundary. To isolate granularity, our study gives coarse baselines exact failure locations and their most favorable native boundaries. Even under these conditions, correct recovery sacrifices independent progress within a turn or checkpoint.

The underlying problem is that execution order does not encode failure dependence. Consider four interleaved steps: s_0 writes a faulty formatter, s_1 uses it to generate a report, s_2 edits an unrelated document, and s_3 tests the faulty output. Recovery should remove $\{s_0, s_1, s_3\}$ while retaining s_2. Undoing only s_0 leaves derived state behind, while restoring a prefix also discards s_2. Changing among turn-level checkpoints, temporal suffixes, and whole branches merely moves this collateral loss across workloads. No fixed coarse boundary consistently avoids it.

Selective recovery raises three systems challenges. First, *dependency discovery* must uncover producer–consumer relations hidden behind shells, compilers, tests, and descendant processes. Second, *object identity* must relate effects across aliases, renames, replacements, and layered filesystems. Third, *selective reconstruction* must rebuild a non-prefix workspace while aligning agent-visible context with that state.

The root cause is a mismatch between how the compared substrates organize recovery state and how failures propagate. Their recovery boundaries follow commands, checkpoints, or branches, while failures propagate through dependencies among versioned effects. The trajectory should therefore remain one speculation unit, but recovery should

select a dependency-defined subgraph of tool-step effects. Recording individual steps alone is insufficient because a temporal suffix still follows execution order rather than dependence.

AGENTTX keeps one speculative environment across tool calls while recording their observed effects and dependencies. When a step fails, it selects the affected steps, tracks the object versions needed to remove them, and reconstructs the workspace and agent context from the same survivor set.

We implemented AGENTTX on Linux with dependency capture, object-aware historical reconstruction, fail-closed alias handling, and conversation rewind. Across 144 controlled runs, AGENTTX retained 100% of independent work while removing 100% of invalid descendants. At 64 calls, temporal recovery retained only 41% of independent work. At 48 document entries, preserving valid artifacts avoided 1,425 replay tokens relative to temporal recovery and 3,340 relative to whole-session abort. These results indicate that recovery granularity is not only an execution-overhead problem: reducing isolation cost cannot recover valid work once a coarse rollback boundary has discarded it. We evaluate the prevalence and end-to-end benefit of this opportunity separately from the controlled recovery mechanism.

This paper makes three contributions:

- We identify the recovery-unit mismatch in stateful coding-agent trajectories and characterize the limits of fixed coarse boundaries.
- We build AGENTTX to recover dependency-selected effects while keeping the workspace and agent-visible context aligned.
- We evaluate AGENTTX across recovery correctness, retained work, runtime, replay cost, object identity, and context consistency.

2 Background

2.1 Stateful Coding-Agent Workspaces

Coding agents execute software-engineering tasks as iterative interactions with an operating-system environment. Systems such as SWE-agent and OpenHands let a model inspect repositories, edit files, invoke shell commands, and run tests [2, 3]. A task therefore spans multiple tool calls rather than one isolated program execution. Each call observes repository state left by earlier calls. Shells may also launch compilers, test runners, package managers, and descendant processes whose effects enter the same workspace. The filesystem is therefore persistent execution state: later calls can consume source edits, generated artifacts, build products, and configuration changes.

The broader agent literature makes this execution pattern explicit. Tool-use controllers learn when to invoke external APIs [12]. Reasoning- and-action loops and self-reflection extend trajectories across feedback steps [13]. Interactive benchmarks evaluate multi-turn state and real websites, as in AgentBench [14] and WebArena [15]. Desktop and stateful-tool suites broaden this setting to OSWorld [16] and ToolSandbox [17]; AppWorld evaluates longer application workflows [18], while BrowserGym targets multi-step web interaction [19].

This execution model makes the workspace boundary important. Direct execution exposes every mutation to the host as soon as a tool runs. Recent agent systems instead keep filesystem changes provisional. YoloFS stages session mutations outside the base filesystem and provides snapshots that an agent can return to during self-correction [6]. OpenHands similarly supports sandboxed execution for software-development agents [3]. These systems let an agent continue operating on a mutable workspace while retaining a recovery point.

Repository-level coding systems add another source of long-lived state. They retrieve context across files, localize faults, edit repositories, and execute tests iteratively; RepoCoder [20] and AutoCodeRover [21] illustrate this workflow. Agentless and HyperAgent extend it to autonomous repair [22, 23], while SWE-gym studies longer interactive trajectories [24]. Recent benchmarks broaden the workload from issue fixes to feature development and software evolution [25]. FeatureBench studies feature-level development [26], while SWE-rebench revisits repository evolution [27]. Other suites cover repository-scale work, research replication, and experiment execution; examples include GitTaskBench [28], PaperBench [29], and MLAgentBench [30].

2.2 Speculation and Recovery Units

Existing substrates generally expose recovery at the boundary of the state container they preserve. `try` introduces a semisolate around an opaque component invocation: it captures the command's filesystem effects so they can be inspected, applied, stacked, or discarded before they modify the live system [5]. Sandlock similarly confines a process invocation and uses a copy-on-write work directory to commit or discard its filesystem effects [31]. Agent-native filesystems extend this lifetime across a session. YoloFS retains staged mutations and uses snapshots as restoration points [6], while BranchFS gives each speculative branch an isolated copy-on-write filesystem view with branch-level commit or abort [7].

hS instead targets script performance: it speculatively reorders black-box subprocesses at fine granularity and tracks dependencies to preserve the script's sequential semantics [32]. It is an execution optimization rather than a post-failure recovery mechanism for agent state.

Checkpoint/restore systems preserve still broader execution state. Crab observes turn-level OS effects to decide when a sandbox checkpoint is recovery-relevant, but restoration still returns the sandbox to a selected checkpoint [33]. Waypoint captures filesystem, process, memory, and termi-

nal state, and materializes a new live fork from a selected temporal checkpoint [8]. DeltaBox accelerates checkpoint and rollback of stateful agent sandboxes by storing changes between consecutive checkpoints; its restored state includes both filesystem and process state [9]. Thus, these systems span command effects, staged sessions, branches, and complete session or sandbox checkpoints. Table 1 summarizes the corresponding speculative and recovery boundaries.

Table 1: **Speculation and recovery units.** Existing agent execution substrates preserve and recover state at command, process, session, branch, sandbox, snapshot, or checkpoint granularity.

System	Speculation unit	Recovery unit
`try` [5]	Command	Command
Sandlock [31]	Process	Process
YoloFS [6]	Session	Snapshot
BranchFS [7]	Branch	Branch
Waypoint [8]	Session	Checkpoint
Crab [33]	Sandbox	Checkpoint
DeltaBox [9]	Sandbox	Checkpoint

For the mechanisms in Table 1, the state abstraction determines both speculation lifetime and the native recovery boundary. They span command, session, branch, and checkpoint state, but do not expose a dependency-defined recovery unit within one trajectory.

3 Observation and Motivation

Recovery mechanisms inherit their rollback boundaries from the execution substrate. We first examine what state each boundary can select. A shared multi-turn microbenchmark measures the valid work each boundary sacrifices after a crash, followed by an audit of opportunities in native agent traces.

Artifact availability prevents a complete implementation-to-implementation comparison. We replay the shared cases through pinned artifacts for `try`, BranchFS, YoloFS, and Waypoint. The first two expose native discard or abort operations rather than post-failure agent recovery. Their results therefore measure the state retained by those native boundaries, not the systems' intent. We represent systems without reusable artifacts by their recovery semantics.

Whole branch is the controlled semantic bound for discarding a complete speculative unit. The checkpoint and temporal baselines are defined in the same way, and we assign no implementation latency to these emulations. Named artifact results remain separate from these bounds.

We deliberately favor every coarse baseline with oracle localization. Each receives the exact failed step, and turn recovery receives the exact affected turns. The baseline then selects the smallest set of native units covering all affected effects. This is the maximum retention that its granularity can express. Detection error, imperfect localization, or implementation constraints can only discard additional valid work.

3.1 Analyzing Recovery Paths of Stateful Agents

Execution-defined boundaries. A coding-agent trajectory $T = \langle s_0, s_1, \ldots, s_n \rangle$ comprises ordered tool steps. Each step executes one tool call against state left by preceding steps. Table 1 groups existing recovery paths by the execution boundary they restore. These boundaries encode chronology or control structure; they do not identify which later steps consumed state produced by the failed step.

Consider a failed producer followed by dependent and independent work in different turns. *Root-only* recovery removes the producer but leaves its dependent descendants. A *turn checkpoint* removes complete turns that contain invalid effects. Because their identities are supplied, it selects no additional turns. A *temporal step* removes the global suffix beginning at the producer, and *whole branch* abandons the complete speculative branch. The latter three paths remove the dependent descendants, but they may also remove independent effects enclosed by their execution boundaries.

We include two controls. *No recovery* preserves every effect, including corrupted state. The *ideal oracle* removes only the failed step and the steps that depend on it, preserving all unaffected work. The oracle represents the desired recovered state without prescribing how to discover it.

3.2 Methodology

Controlled cases. We use one filesystem-effect microbenchmark family. A turn contains 16 tool calls, each represented by one recoverable ledger step; a branch contains one to ten turns. The generator varies turn count, fault position, dependency value, graph shape, and interleaving. We analyze 400 paired manifests: 100 vary task length, 90 vary fault position, and 210 vary dependency value. Every policy receives the same manifest and fault root in each paired case.

The generated dependencies identify the failed root and every step that depends on it. Recovery is complete only if it removes all these affected steps. Among complete policies, collateral cost counts removed steps outside this set. We also attribute each step a deterministic token weight to compare the amount of useful work discarded; this weight is not billed model usage or measured replay. We replay all 400 manifests through `try`, BranchFS, YoloFS, and Waypoint to compare recovered states, not cross-environment latency. The first two observations use this shared population.

Native traces. For the third observation, we inspect fault-free SWE-agent runs on SWE-Bench and Terminal-Bench. We include complete traces regardless of verifier outcome; an unsuccessful task attempt still supplies observed tool steps. We treat each workspace-mutating step with a later step as a hypothetical fault root. The analysis connects accesses to the last preceding writer of each included workspace path. Opaque agent-context steps conservatively link earlier observations to later actions. We compare the re-

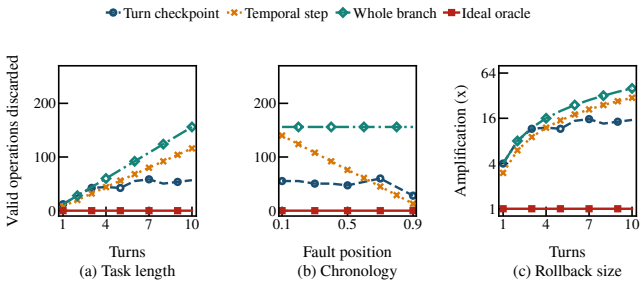


Figure 1: **Correct recovery can still discard avoidable work.** *All plots compare the four correctness-qualified policies. In (a), waste grows with the number of turns for a fixed four-step affected set. In (b), a temporal suffix changes with fault position at fixed task length and affected set. Plot (c) reports rollback amplification. The ideal oracle is an analytic bound, not an implemented recovery policy.*

sulting dependent steps with the temporal suffix that would be removed from the root onward.

We retain tasks and roots with zero observed opportunity in the denominators, and report both per-task incidence and per-root counts. A dependency edge spanning at least five turns is a descriptive slice, not a condition for an opportunity: even a short dependency can leave independent work inside a rollback boundary. This path-level counterfactual excludes some build artifacts and unobserved state. It injects no crash, does not certify that apparently independent steps are independent through agent context, and measures no replay tokens.

3.3 Analyzing Recovery Overheads

Figure 1 exposes the controlled scaling behavior. Its plots retain only policies that remove every affected effect. They hold the ground-truth affected set fixed while varying branch length or fault position, so changes in waste arise from the recovery boundary rather than a larger failure.

With a fixed four-step affected set, increasing the branch from one to ten turns raises discarded valid operations from 12 to 56.8 for turn recovery. Temporal recovery rises from 8 to 116, while branch recovery rises from 12 to 156. The corresponding ten-turn rollback amplification is 15.2×, 30×, and 40×. Moving the fault from 10% to 90% of a fixed ten-turn branch reduces temporal waste from 140 to 13 valid operations. These controlled trends establish that execution structure, rather than affected-set size, drives coarse recovery cost.

Observation #1: Correct coarse recovery still discards independent work. The comparison exposes a correctness–retention tradeoff. Policies that retain nearly all work leave invalid state behind, while correct coarse policies delete valid effects enclosed by their recovery boundaries.

Figure 2 quantifies this tradeoff over all 400 paired manifests. No recovery removes 0% of the affected steps, while root-only undo removes 17.03% on average. Their low rollback cost therefore reflects under-recovery, not efficiency.

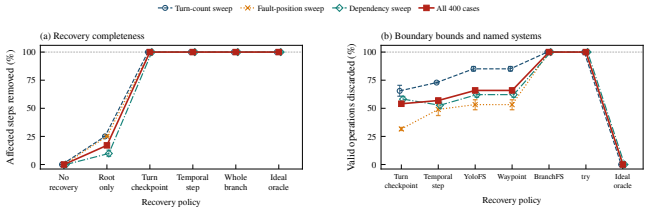


Figure 2: **Recovery tradeoff across 400 paired cases.** *Lines report the turn-count (100 cases), fault-position (90 cases), dependency-value (210 cases), and combined-suite means; error bars give 95% bootstrap confidence intervals. The left plot applies the correctness gate to boundary policies. The right plot compares optimistic boundary upper bounds with artifact-native* `try`*, BranchFS, YoloFS, and Waypoint, each labeled by system name.*

Turn checkpoint, temporal step, whole branch, and ideal recovery remove 100% of the affected set in every case.

Among the semantic policies, checkpoint, temporal, and branch recovery discard 54.02%, 56.90%, and 100% of valid work on average. The ideal boundary discards none. Artifact-native YoloFS and Waypoint each discard 65.89%, 11.87 percentage points more than the oracle-aided turn boundary. BranchFS and `try` discard the complete speculative unit through their native operations. The recovered states agree within each pair across all 400 manifests. Correctness therefore does not imply efficient recovery: coarse boundaries restore safety by deleting independent progress. We first test completeness, then compare the valid work preserved by complete policies.

The turn policy already uses a favorable boundary: it rolls back only turns that contain affected steps. Its 54.02% collateral loss therefore comes from valid steps inside those turns. Substantial avoidable waste remains within the turn, below the finest boundary exposed by turn-level recovery.

Observation #2: Temporal recovery wastes work because time and dependency order diverge. The desired recovered state is usually not a temporal prefix.

Figure 3 exposes this mismatch directly. Coarse recovery follows execution order, whereas failure propagation follows dependencies. When valid and affected effects are interleaved, a temporal boundary cannot separate them. Across the 400 paired manifests, only 10 exact survivor states are temporal prefixes. In the other 390 (97.5%), no single checkpoint removes every affected effect while retaining every valid effect.

To vary the severity of this mismatch, let r be the failed step and S the steps after it. Dependency value is the fraction of S that depends on r. A value of zero means no later step depends on the failure; one means every later step does. The figure sweeps this value from 0 to 1 in a five-turn, 80-operation branch. The exact target is non-prefix in 200 of these 210 cases; only the ten fully dependent cases form a suffix.

At dependency value 0, checkpoint, temporal, and whole-branch recovery discard 660, 2,596, and 3,476 units of useful token weight. They retain 81.01%, 25.32%, and 0% of

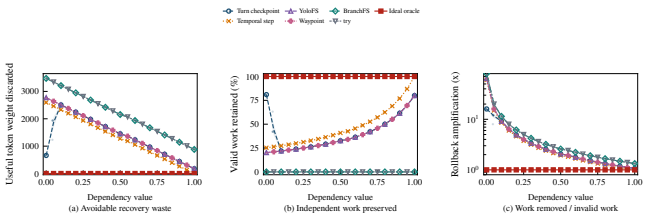


Figure 3: **Recovery waste reflects temporal–dependency mismatch.** *Across all 21 dependency values and ten paired manifests per point, sparse dependencies create the largest gap between execution order and failure dependence.* `try`*, BranchFS, YoloFS, and Waypoint are artifact-native and labeled directly. YoloFS overlaps Waypoint, while BranchFS overlaps* `try`*; distinct markers identify both measurements. Other lines are optimistic boundary semantics. Lines show means, faint points show paired cases, and shaded bands give 95% paired-bootstrap confidence intervals. Token weight is deterministic work attribution, not billed model usage.*

valid work, with rollback amplification of 16×, 60×, and 80×. Artifact-native YoloFS and Waypoint recover identical states in all 210 dependency cases. At dependency value 0, each discards 2,772 units, retains 20.25%, and incurs 64× amplification. BranchFS and `try` match whole-branch recovery because their native operations discard the complete unit. At dependency value 1, YoloFS and Waypoint each discard 176 units and retain 80%. Temporal recovery becomes exact, checkpoint discards 176 units, and BranchFS and `try` each discard 880. The ideal oracle remains at zero useful work discarded, 100% retention, and 1× amplification across the sweep. Denser checkpoints or faster branch aborts can reduce mechanism overhead, but they cannot represent a non-prefix survivor state. Eliminating this waste requires selecting recovery state by dependencies rather than by execution time.

This result locates a granularity gap. Mechanisms that preserve a long-running agent workspace recover at a turn, checkpoint, branch, or session boundary. Command-scoped semisolates expose one call's effects but end the speculative unit at that call. A long-lived runtime must instead retain the effects of individual tool steps across the trajectory. Step-level recording is necessary, but the next observation shows that it is not sufficient.

Observation #3: Long-range dependence alone does not imply a large recovery opportunity. We next inspect fault-free agent trajectories to test whether the structural opportunity constructed above appears in real executions. This analysis treats each observed workspace write as a hypothetical fault root; it does not inject a crash or measure replay tokens. We use SWE-agent because the current Codex CLI exporter groups its entire execution into one turn, preventing a cross-turn comparison.

Figure 4 separates long-distance dependencies from independent work that a temporal suffix would discard. Cross-turn workspace dependencies occur in 171 of 189 complete SWE-Bench traces and 44 of 54 Terminal-Bench traces. Edges spanning at least five turns appear in 99 and 15 traces.

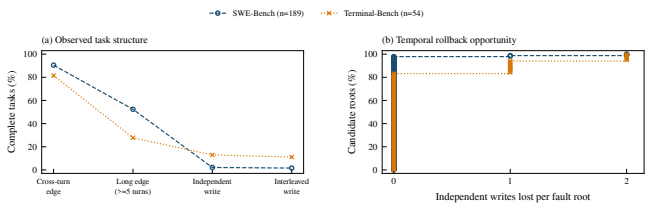


Figure 4: **Observed recovery opportunities in native SWE-agent traces.** *This interim full-sweep snapshot contains 189 complete SWE-Bench and 54 complete Terminal-Bench traces, including failed task attempts. A long edge spans at least five turns; an interleaved write lies outside the observed dependency closure but between affected steps. Panel (b) considers 236 and 101 hypothetical write-root candidates, respectively. These are observed-path counterfactuals, not certified independent work or measured replay cost.*

Yet only 4 and 7 traces, respectively, contain an observed later independent write; just 3 and 6 place such a write inside the affected execution interval. Correspondingly, 231 of 236 SWE-Bench candidate fault roots and 84 of 101 Terminal-Bench roots have no additional independent write for temporal recovery to discard.

Thus real trajectories exhibit the temporal–dependency mismatch, but a long dependency span is not itself evidence of large avoidable loss. The observed opportunity is sparse and workload-dependent. Our path analysis excludes some build artifacts and cannot certify independence through agent context or unobserved process state; the figure bounds a possible opportunity rather than demonstrating realized savings. The controlled cases above isolate the mechanism, while real-agent recovery must establish its end-to-end benefit separately.

Significance. Together, the observations progressively narrow the required recovery model. Observation #1 excludes policies that appear efficient by leaving invalid state behind and exposes substantial collateral loss within affected turns. Observation #2 attributes that loss to disagreement between temporal order and dependencies. Observation #3 shows that long-range dependencies occur in native trajectories, while the independently writable work that could be retained varies sharply across tasks. The required recovery unit is therefore a dependency-selected set of tool-step effects, not another fixed interval in execution time.

3.4 Motivation

Attempts. The three observations rule out refining a temporal boundary as a complete solution. No recovery and root-only undo fail the correctness gate. Turn checkpoints and branch abort remove invalid state but discard valid work inside their boundaries. Temporal-step recovery uses a finer unit, yet still removes a suffix. The native-trace audit shows that the possible benefit depends on whether independent work actually overlaps the affected execution interval.

Insight. The observations show that a turn is too large to serve as the recovery unit. Each tool step and its OS ef-

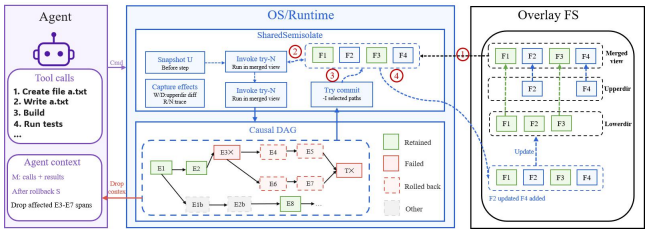


Figure 5: **Execution and selective recovery in AGENTTX.** *The agent executes tool calls against a shared overlay view. The runtime records their effects and dependencies, removes the failed step and its descendants from the workspace and agent context, and retains independent work.*

fects should instead be recorded separately. Step granularity alone is insufficient: recovery must select these records by dependency rather than remove a temporal suffix. The trajectory remains the speculation unit, while a dependency-defined subgraph of step effects becomes the recovery unit. This separation preserves independent effects on both sides of a failed step.

Design challenges. Realizing selective recovery proposes three challenges: 1) *dependency discovery*, uncovering relations hidden inside shells, compilers, tests, and descendant processes; 2) *object identity*, tracking objects through aliases, renames, replacements, and layered filesystems; and 3) *selective reconstruction*, rebuilding a state that may never have existed at a checkpoint and aligning the agent-visible context with it. The next section states the recovery model used to describe these requirements.

4 Selective Recovery Model

To describe the separation observed in Section 3, we model the trajectory as an *Agent Effect Transaction* (AET). This notation gives execution and recovery different units: the trajectory remains speculative, while recovery selects a dependency-defined subset of versioned effects. The same selection applies to the workspace and agent-visible context. Section 5 maps this model to Linux mechanisms in AGENTTX.

4.1 Overview and Design Principles

Execution and effect capture. Figure 5 connects agent tool calls and context (left), the runtime (center), and the overlay filesystem (right). Before a call, AGENTTX snapshots the upper layer, then runs the command through `try` in the merged view. In the example, the merged view takes F1 and F3 from the lower layer while the upper layer updates F2 and adds F4. After the call, an upper-layer diff records writes and deletes; tracing records reads and failed lookups.

Dependency-guided recovery. The captured effects form a DAG of tool steps. In the figure, failed E3 invalidates E4–E7, but later E8 remains on an independent path. AGENTTX rebuilds the overlay view without the invalid effects and removes their associated calls and results from the agent context. E8 and its context remain available. The separate `try` commit path publishes selected surviving files to the host; it does not determine the rollback set.

Design principles. An AET spans the complete tool-call trajectory rather than one command or recovery attempt. It records dependencies from observed reads, absence, writes, and deletes rather than inferring them from command text. Recovery reconstructs object versions without the selected steps while preserving supported retained effects. The model-visible history follows the same survivor set.

4.2 Recovery State and Transitions

Scope. We model workspace-local filesystem effects and the agent-visible tool-call history produced by trusted but fallible coding agents. Network services, hidden provider state, process memory, terminal state, and adversarial confinement remain outside the transaction.

State. An AET is the tuple $\langle V, L, H, M \rangle$. V is the current speculative filesystem view. L records tool calls, typed effects, dependencies, and status. H stores historical object states needed for reconstruction. M records agent-visible message spans and their associations with ledger steps.

The tuple assigns one role to each recovery concern. V provides cross-call workspace state. L determines causal recovery sets. H supplies versions that can remove selected effects. M determines which tool results remain valid after rollback.

Transitions. *Append* executes one tool call in V, appends its effects to L, and binds the corresponding request and result in M. *Recover* computes a causal closure, reconstructs the affected portion of V from H, and rewinds M to the matching survivor set.

Invariants. Every successful recovery preserves two properties. *Selective recovery* removes every selected effect while preserving supported, non-overlapping retained effects. *Workspace–context agreement* removes invalid tool results from M while retaining messages associated with surviving effects.

These semantics do not require a particular tracer, snapshot representation, or provider message API. The remaining subsections define the recovery set and the reconstruction of workspace and agent-visible context.

4.3 Causal Recovery Semantics

Effect records. Each ledger node $s_i = (i, \text{tool}_i, E_i, P_i, \text{status}_i)$ represents one tool step. E_i contains its observed filesystem effects and P_i contains the identifiers of earlier steps on which it depends through either workspace state or an agent-visible result. Effects include successful reads (R), negative lookups (N), writes (W), and deletes (D).

Dependency rule. Let $\text{overlap}(p, q)$ hold when two effect names identify overlapping filesystem objects. For an active earlier step $j < i$, the transaction adds $j \in P_i$ when:

- s_j writes or deletes p, and s_i reads, negatively looks up, writes, or deletes an overlapping q; or

- s_j negatively looks up p, and s_i later writes an overlapping q; or
- the request for s_i consumes a value returned to the agent by s_j.

The write/delete rule captures dependencies induced by observed or subsequently modified state, including read-after-write and write serialization. The negative-lookup rule captures dependencies on absence: a later creation can invalidate namespace state observed by an earlier negative lookup. The final rule captures values that flow through the agent context without appearing in a filesystem read.

Object relation. Path equality alone does not define overlap. We make object identity explicit with an abstract relation $\equiv_I$ over effect names. Path equality implies overlap, while aliases may also overlap when the identity relation can establish that they refer to the same object.

A descriptor read is associated with the object opened earlier only when observation supplies that relation. Otherwise, the effect remains unknown instead of receiving a guessed dependency. Selective recovery that requires an unresolved identity relation must fail closed and lies outside the recovery guarantee. The identity relation is part of the recovery model; Section 5 describes how AGENTTX approximates it on Linux.

Recovery set. For a failed active step f, the transitive descendant closure is

$$\begin{aligned} C_0(f) &= \{f\}, \\ C_{k+1}(f) &= C_k(f) \cup \{s_i \mid P_i \cap C_k(f) \neq \emptyset\}, \\ C(f) &= \bigcup_{k \geq 0} C_k(f). \end{aligned} \tag{1}$$

$C(f)$ is the least transitive descendant closure of f among active steps. It may contain steps that are separated in time by independent work. Temporal rollback instead selects the contiguous suffix $\{s_i \mid i \geq f\}$, regardless of whether each later step depends on f.

4.4 Object-Aware Reconstruction

Target version. Computing $C(f)$ identifies which effects should no longer contribute to the speculative state, but removing their latest writes is not sufficient when the same object has been modified repeatedly. Recovery must instead reconstruct the version that would remain after excluding the selected recovery subgraph.

Let $W(C)$ denote the objects written or deleted by steps in $C(f)$. For each target in $W(C)$, recovery obtains from H a historical state that predates its earliest selected effect. The retained active history is then considered against the selected targets so that later independent state is not discarded solely because it occurs after the failed step.

Retained overlap. A retained effect can nevertheless conflict with selective reconstruction. If a retained active effect overlaps a target in $W(C)$ and the model cannot establish a supported reconstruction, recovery rejects the operation rather than guessing a merge. This fail-closed rule trades availability for the following safety invariant:

Recovery guarantee. After successful recovery within the stated identity and reconstruction contract, no write or deletion from $C(f)$ remains visible. Every supported, non-overlapping retained effect remains visible in the speculative view.

Only after reconstruction succeeds are the selected ledger nodes marked as rolled back. Reconstruction therefore changes the workspace by causal membership rather than restoring a complete temporal snapshot. Unreachable historical states may then be reclaimed.

4.5 Agent-Context Reconstruction

Message association. M groups each tool request and result into a span associated with its ledger step. If a later request copies a value from an earlier result, the runtime adds that relation to P_i, even when the value never enters the filesystem.

Shared recovery set. The closure $C(f)$ follows both filesystem and message dependencies to a fixed point. The runtime removes the filesystem effects and message spans of exactly those steps, plus stale recovery-control messages, while retaining independent steps on both sides of the failure.

Reconstructed view. The next model request is assembled only from active spans and an authoritative notice that identifies retained and invalidated effects. The reconstructed workspace and model-visible history therefore describe the same survivor set. Hidden provider state is outside this guarantee; black-box agents receive a retained-effects manifest instead of an in-place message rewrite.

5 AgentTX Implementation

AgentTX maps each model component to a Linux mechanism. A shared OverlayFS view keeps the trajectory in V; effect capture constructs L; the version store and object catalog preserve H; a ledger-bound conversation log maintains M. Together, these mechanisms preserve trajectory-wide speculation while restricting rollback to a dependency-defined effect subgraph. We follow this path from execution through fault localization to workspace and context recovery, then describe performance optimizations.

5.1 Runtime Architecture and Continuous Speculation

Workspace construction. For each tool call T_i, AgentTX selects the project baseline as the read-only `lowerdir`, creates the session's writable `upperdir`, and allocates the OverlayFS `workdir` and `merged` mount point. The command runs against `merged`, not the host project.

OverlayFS resolves reads against the upper layer and then the baseline. A write to a lower object triggers copy-up, a new object enters the upper layer, and deletion creates a whiteout. The kernel uses `workdir` for internal bookkeeping. AgentTX configures these directories and controls their lifetime; OverlayFS performs lookup, copy-up, and whiteout

operations.

Cross-call state. All calls in a session reuse one `try -N` upper layer. Later calls therefore observe earlier provisional effects, while the base workspace remains unchanged. The launcher synchronizes `$PWD` with the protected workspace so the process working directory and environment refer to the same view.

The runtime now has one call's effects inside the shared trajectory, but not their cross-call dependencies. Ledger construction adds the edges that define the recoverable subgraph.

5.2 Effect Capture and Ledger Construction

Mutation effects. AgentTX fingerprints the unmounted upper layer before and after each command. Content and metadata fingerprints capture writes, chmod, chown, touch, empty directories, symlinks, and deletions. OverlayFS character-device and `.wh.*` whiteouts are both normalized to *D* effects.

Observation effects. AgentTX uses `strace` [34] to capture successful reads and ENOENT/ENOTDIR lookups. The parser follows subprocess creation and working directory changes. A failed trace or parse aborts the call unless observation is explicitly disabled. Trusted tools may instead declare effects, making the declaration part of the trusted computing base.

Path resolution and coverage. The parser resolves supported pathname operations relative to the current directory or a known file-descriptor origin. It retains both requested and resolved names when a symlink changes the path. A non-`AT_FDCWD` operation without a known descriptor origin fails closed. Modeled Linux pathname syscalls define the coverage boundary; the trace does not prove that every possible filesystem access was observed.

Ledger construction. At command return, AgentTX merges the observed *R* and *N* effects with the derived *W* and *D* effects. It normalizes paths and supported identities, preserves observable order within the call, and appends one record E_i. The ledger then compares E_i with active earlier records using the rules in Section 4.3. Unsupported identities remain unknown rather than receiving dependencies inferred from command text.

5.3 Historical Versions and Object Identity

Version store. Before step *i*, the layer store records `before_i`; after the call, it records the resulting upper-layer state as V_i. Regular-file contents are stored as immutable content-addressed blobs and linked into snapshots. Native whiteouts, directory metadata, and mode-000 trees are preserved. These states provide the pre-effect versions required by Section 4.4.

Namespace normalization. AgentTX treats ancestor-related paths as overlapping and retains lexical and resolved names for supported symlink accesses. A rename contributes a delete at the old name and a create at the new name. These rules cover namespace relations that can be established from the observed operation.

Hard-link identity. The active OverlayFS path enables `index=on` to preserve hard-link visibility during live copy-up. A persisted object catalog connects accesses through known sibling names and records tested upper-created link groups. Recovery expands a known alias group before checking overlap or reconstructing an object.

Identity boundary. AgentTX promotes a hard-link relation only when the substrate or upper-layer materializer verifies it. Bind mounts, external aliases, arbitrary generation changes, and general multi-object identity remain outside the contract. An operation that requires one of these relations fails closed.

5.4 Fault Localization and Recovery Planning

Failure observation. A failed verifier establishes that the current view is invalid, but does not identify the faulty step. AgentTX records the failed check as a ledger step and uses its observed dependencies to find earlier active steps that changed state. The search excludes committed steps and, when available, steps before the last passing check. Dependency distance and diagnostic references rank candidates; neither constitutes proof that a candidate is the fault's origin.

Counterfactual checks. For each candidate, AgentTX forks the speculative view, removes the candidate and its dependents, and reruns the same verifier in the fork. It can also test candidate groups to handle interacting faults. Among plans that pass within the probe budget, it selects one that discards the least active work, with conversation loss as a secondary cost. A passing check establishes a sufficient recovery plan for that verifier, not a unique root cause. If no plan passes, localization is inconclusive and the causal path does not change the live view. The real-workload causal evaluation disables optional coarse fallback so an inconclusive search cannot count as causal recovery.

5.5 Causal Reconstruction

Recovery plan. AgentTX applies a verified plan to the live view only after counterfactual checks finish. For its selected root steps, it alternates ledger and conversation expansion until the recovery set reaches a fixed point. It collects objects written or deleted by that unified closure, expands known alias groups, and chooses the version preceding each object's earliest selected effect.

Retained-state check. The runtime compares every target with active effects outside the closure. A non-overlapping retained effect can survive. An overlap without a supported object-level reconstruction rejects recovery before the speculative view is changed.

Reconstruction. AgentTX builds a fresh upper generation from the selected historical versions and rebases retained state outside the closure onto it. It validates every affected target before swapping the generation into the live session. Ledger nodes become rolled back only after the

swap succeeds. Recovery reconstructs the view from effect history; it does not replay the trajectory.

5.6 Conversation Reconstruction

Ledger-bound spans. AgentTX persists each native-agent tool request and result as a call span bound to its ledger step. The conversation log also records message dependencies when a later tool request copies a distinctive value from an earlier result.

Synchronized rewind. Workspace reconstruction and conversation rewind consume the same fixed-point recovery set. After reconstruction succeeds, the runtime invalidates the corresponding spans and stale control messages, then rebuilds the next provider request from active spans.

Recovery handoff. Native tool-calling agents resume from the reconstructed message list plus a ledger-derived notice naming retained and invalidated state. External black-box agents cannot expose mutable provider history, so they resume from a compact retained-effects manifest. AgentTX does not claim to restore hidden reasoning state or process memory.

5.7 Performance Engineering

Persistent execution. A Python worker exchanges length-prefixed requests inside one semisolate. This amortizes namespace and overlay setup across a long trajectory. If the worker dies, the current request uses the one-shot path and the next request starts a new worker. Worker reuse does not change recovery semantics.

Incremental history. After the initial snapshot, AgentTX clones the previous snapshot and replays only paths changed by the latest call. Content-addressed blobs deduplicate regular-file contents across versions. These optimizations reduce history maintenance while preserving the same H states used by reconstruction.

6 Evaluation

Our evaluation asks four questions, separating controlled recovery behavior, real-agent task outcomes, component contributions, and runtime overhead:

RQ1 On matched multi-turn workloads, does AgentTX remove invalid effects, retain independent work, and reduce repair cost while keeping workspace and context consistent (Section 6.2)?

RQ2 On official coding and terminal tasks, how do recovery policies affect task success, total model tokens, and end-to-end time (Section 6.3)?

RQ3 Which dependency-capture, object-identity, reconstruction, context, and execution mechanisms explain correctness and cost (Section 6.4)?

RQ4 What execution, tracing, history-storage, and recovery overheads does AgentTX introduce as trajectories grow (Section 6.5)?

Table 2: **SWE-agent recovery on all 300 SWE-Bench Lite tasks.** *Success uses all 300 planned tasks as the denominator. Mean tokens include successful and failed complete runs, and include replay. The clean reference has 299 complete runs and one worker timeout.*

Policy	Success /300	Mean tokens per complete run	Replay tokens all complete runs
AgentTX	285 (95.00%)	135,004	0
Ideal recovery	289 (96.33%)	133,251	0
Clean reference	273 (91.00%)	154,355	0
No rollback	282 (94.00%)	311,673	0
Temporal checkpoint	284 (94.67%)	159,201	1,954,855
Whole-branch rollback	243 (81.00%)	160,053	2,898,444

Table 3: **Token savings from causal recovery on matched successful tasks.** *Each row pairs the same tasks successfully completed by AgentTX and the indicated policy. Total savings are* $1 - \sum T_{\text{AgentTX}} / \sum T_{\text{control}}$*; median savings are the median of per-task savings. Both include main and replay API tokens.*

Comparison	Tasks	Total savings	Median savings
No rollback	275	58.43%	55.60%
Temporal checkpoint	278	17.55%	18.91%
Whole-branch rollback	239	27.86%	32.31%
Ideal recovery	282	1.47%	−2.25%
Clean reference	264	14.38%	11.36%

6.1 Setup and Methodology

6.2 RQ1: Controlled Recovery and Repair Cost

6.3 RQ2: Real-World Workloads

SWE-agent on SWE-Bench Lite. We evaluate all 300 SWE-Bench Lite tasks with six recovery policies, one run per task and policy, using SWE-agent with GPT-5.6 Luna. The campaign has 1,800 planned cells: 1,799 have complete results and one clean-reference cell ends in a worker timeout. A successful run passes the target `FAIL_TO_PASS` tests and the applicable recovery-state, completion, and commit checks. AgentTX, ideal recovery, and the clean reference use compact context; no rollback, temporal checkpoints, and whole-branch rollback use full context. All token measurements below use API-reported usage and include both the main repair session and any separate replay session.

Figure 6 and Table 2 report task outcomes and measured model cost. AgentTX succeeds on 285 tasks, compared with 282 for no rollback, 284 for temporal checkpoints, and 243 for whole-branch rollback. The ideal recovery reference succeeds on 289 tasks. Across all complete runs, AgentTX uses 40,501,138 tokens, compared with 93,502,009, 47,760,194, and 48,015,838 for the three controls, respectively. The clean worker timeout additionally incurs 1,958,229 known tokens from returned API responses; usage for its last in-flight request is unknown and is excluded from the complete-run mean.

Recovery cost. On matched successful tasks, causal recovery reduces total model tokens by 58.43% relative to no rollback, 17.55% relative to temporal checkpoints, and 27.86% relative to whole-branch rollback (Table 3). The respective 95% task-paired bootstrap intervals are 53.42–62.82%, 11.20–23.48%, and 22.70–32.91%. The savings persist on the fixed 212 tasks completed successfully by all

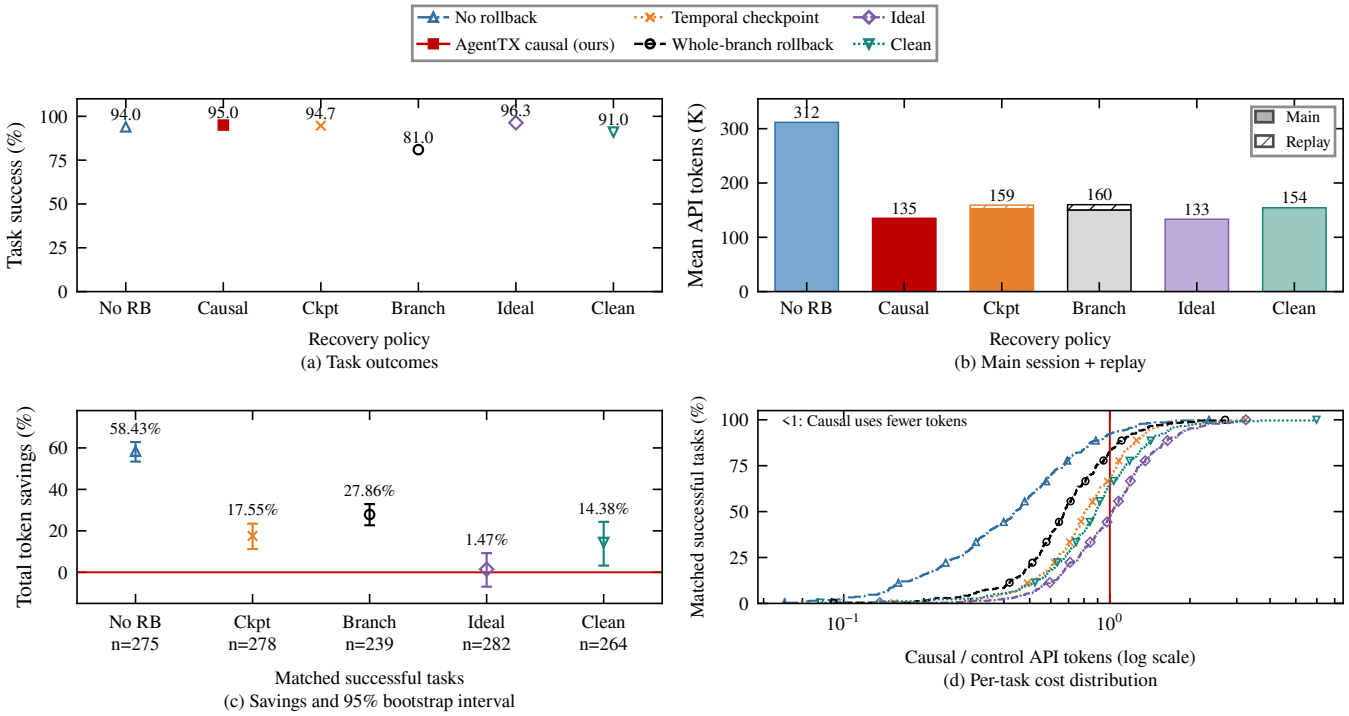


Figure 6: **Causal recovery on the full SWE-agent / SWE-Bench Lite campaign.** *(a) Success over 300 planned tasks per policy. (b) Mean main-session and replay API tokens over all complete runs, including task failures (299 runs for Clean, 300 for each other policy). (c) Total token savings on matched successful tasks with 95% task-paired bootstrap intervals. (d) The empirical distribution of per-task Causal/control token ratios on the same successful pairs; ratios below one favor Causal.*

six policies: 60.02%, 17.25%, and 26.64%, respectively. Removing the single largest absolute token-difference task in each comparison as a sensitivity check leaves savings of 58.07%, 15.97%, and 27.20%; the primary estimates retain every matched task. Thus the observed cost reduction is not explained by one extreme task. AgentTX requires no replay session; temporal checkpoints and whole-branch rollback incur 1,954,855 and 2,898,444 replay tokens across their complete runs.

6.4 RQ3: Breakdown

6.5 RQ4: Other Overheads

7 Discussion and Limitations

Why not make causal rollback the default? The active view uses OverlayFS `index=on`, which preserves hard-link visibility within the tested session [35]. Recovery expands verified pre-existing and upper-created alias groups before reconstruction. Causal recovery still fails closed for arbitrary multi-object topology, bind mounts, external aliases, and unsupported generation changes.

Why trace instead of infer commands? Command strings are not a reliable specification of transitive compiler, test, or script reads. System-call tracing observes actual accesses but adds workload-dependent cost and remains Linux-specific. We therefore use the strace path for the main correctness and performance claims. In the measured 12-step comparison, it costs 25.60 ms/step versus 61.39 ms/step for the optional persistent eBPF backend. Both capture the same read and negative-lookup cases in this probe.

The eBPF path remains a replaceable implementation, not a claim of lower overhead. Neither tracer removes the need to define alias and unsupported-syscall semantics or repairs an inode relation already split by OverlayFS copy-up.

What state is outside the transaction? The current AgentTX prototype places workspace-local filesystem effects and ledger-bound tool-call history inside its transaction state. Hidden provider reasoning, network requests, emails, package registries, cloud control planes, process memory, and terminal state remain outside the guarantee. These effects may be irreversible or require subsystem-specific compensation; systems that capture full process or session state are complementary to AGENTTX.

What is outside the recovery scope? AgentTX determines and reconstructs the surviving speculative state. Publishing that state to a user workspace, coordinating concurrent publishers, and making publication crash-consistent are separate concerns outside this paper.

How strong is the current evidence? The native audit covers 20 successful trajectories and 638 turns, but only six turns pass the independence gate. The real-turn result is therefore a narrow counterfactual, not an end-to-end success study. Controlled motivation traces reach 128 tool-call steps,

while failure/reload tests reach 256 steps. The matched strategy study uses 30 Codex cells and 25 native-SWEAgent cells per policy. It uses v8 for no rollback and retained v7 measurements for the other policies. The study therefore supports within-backend comparisons rather than paired protocol-version or cross-backend claims.

The historical application check uses six SWE-Bench Lite and Terminal-Bench tasks with one oracle-repair run per policy. It does not establish live-agent application success. The 300-task, six-policy SWE-agent campaign now has 1,799 complete results and one clean-reference worker timeout. It measures live repair and recovery-contract success with target `FAIL_TO_PASS` tests, as reported in Section 6.3. The earlier live-agent control-plane probes remain seeded and use one model.

`try`, BranchFS, YoloFS, and Waypoint run artifact-natively on all 400 motivation manifests. `try` and BranchFS exercise native whole-unit discard; YoloFS and Waypoint exercise snapshot restoration. Waypoint runs in a privileged container, so these measurements do not support cross-environment latency claims. Sandlock builds on this host, but the kernel does not enable Landlock. Systems without a reusable recovery artifact are represented only by optimistic boundary semantics. We do not attribute those proxies to an external implementation.

8 Related Work

AgentTX is closest to effect-capturing semisolates, agent-oriented filesystems, and checkpoint or branch runtimes. Its distinguishing recovery unit is a dependency-defined effect subgraph within one speculative trajectory. We compare that semantic choice using matched recovered states, while keeping runtime claims separate across host and container environments.

Effect capture and semisolates. `try` introduces unprivileged semisolates for opaque commands, including effect introspection, optional application, and stacking [5]. Sandlock confines a single process run and makes its filesystem effects reversible through a copy-on-write work directory [31]. AGENTTX uses `try`'s mechanisms as its execution substrate. AgentTX extends this substrate from isolated calls to one speculative trajectory, then recovers a dependency-defined effect subgraph through historical reconstruction and context-aligned rollback.

Agent-native filesystems and branches. YoloFS stages mutations, exposes snapshots, and uses progressive permission to improve agent/user control [6]. Its filesystem mediation and permission interface are broader than AGENTTX's current substrate. BranchFS provides nested copy-on-write branch contexts with atomic leaf commit/abort and first-commit-wins coordination [7]. These systems select snapshots or branches. AgentTX instead selects a dependency-defined effect subgraph inside one evolving trajectory.

Checkpoint and restore. Waypoint combines filesystem deltas with process, memory, shell, and terminal state for branchable sessions [8]. DeltaBox accelerates complete sandbox checkpoint/rollback using incremental filesystem and process state [9]. Crab observes turn-level effects to avoid checkpoints that do not contain recovery-relevant state [33]. Their state coverage or checkpoint efficiency is complementary to AGENTTX. AGENTTX instead answers which non-contiguous filesystem effects to remove after a failure has been identified.

Process checkpointing. CRIU and DMTCP preserve complete process/session state for restart, migration, or debugging [10, 11]; their recovery unit is an application image, not a selected filesystem effect subgraph. AgentTX instead keeps the existing unmodified application interface, infers effects at opaque tool boundaries, and preserves valid later work after a selective failure.

Filesystem topology and tracing. OverlayFS specifies copy-up, whiteout, and index semantics that determine when two pathnames still denote one object [35]. System-call tracers such as `strace` observe accesses made by arbitrary descendants [34]. The resulting paths still need hierarchy, descriptor, and alias normalization. Our contribution connects that observation layer to a dependency ledger and a fail-closed reconstruction policy. A different tracer alone does not solve object identity or selective rollback.

The surrounding systems literature exposes complementary boundaries. MicroVM and userspace-kernel sandboxes study isolation cost [36, 37], while provenance systems record whole-system data and control flow [38]. Kernel tracing work uses eBPF for low-overhead event capture and distributed path reconstruction [39, 40]. These mechanisms can improve observation, but they do not select the rollback set or align model context with the reconstructed workspace.

9 Conclusion

Long-running coding agents need a speculation unit larger than one command and a recovery unit smaller than one session. AGENTTX keeps the trajectory speculative while recovering a dependency-defined subgraph of versioned effects. It combines effect capture, object-aware history, selective workspace reconstruction, and context-aligned rollback.

Across 144 controlled runs, it retained all independent work and removed all invalid descendants. At 48 document entries, it avoided 1,424.7 replay tokens over temporal recovery and 3,340.3 over whole-session abort. These claims cover supported workspace-local filesystem effects with fail-closed object identity. The benefit scales with dependency sparsity rather than arising in every turn.

References

[1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models.

In *International Conference on Learning Representations (ICLR)*, 2023.

[2] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[3] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, et al. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025.

[4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[5] Evangelos Lamprou, Tianyu (Ezri) Zhu, Di Jin, Grigoris Ntousakis, Georgios Liargkovas, Calvin Eng, Konstantinos Kallas, Michael Greenberg, and Nikos Vasilakis. Controlling Opaque-Component effects with semisolates and try. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pages 453–471. USENIX Association, 2026.

[6] Shawn Wanxiang Zhong, Junxuan Liao, Jing Liu, Mai Zheng, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Don't let ai agents yolo your files: Shifting information and control to filesystems for agent safety and autonomy, 2026.

[7] Cong Wang and Yusheng Zheng. Fork, explore, commit: Os primitives for agentic exploration, 2026.

[8] Jiakai Xu, Tianle Zhou, Danielle Gillai, Ruizhe Fu, Georgios Liargkovas, Patrick Shen, Kostis Kaffes, and Eugene Wu. Toward systems foundations for agentic exploration. In *Systems for Agentic AI Workshop at SOSP*, 2025.

[9] Yunpeng Dong, Jingkai He, Shiqi Liu, Yuze Hou, Dong Du, Zhonghu Xu, Si Yu, Baochuan Yang, Yubin Xia, and Haibo Chen. Deltabox: Scaling stateful ai agents with millisecond-level sandbox checkpoint/rollback, 2026.

[10] CRIU developers. Checkpoint/restore in userspace, 2026.

[11] Jason Ansel, Kapil Arya, and Gene Cooperman. DMTCP: Transparent checkpointing for cluster computations and the desktop, 2009.

[12] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, et al. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[13] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[14] Xiao Liu, Hao Yu, Hanchen Zhang, et al. Agentbench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024.

[15] Shuyan Zhou, Frank F. Xu, Hao Zhu, et al. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024.

[16] Tianbao Xie, Danyang Zhang, Jixuan Chen, et al. OSWORLD: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[17] Yifan Zhang, Chenglei Si, et al. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities, 2024.

[18] Harsh Trivedi, Tushar Khot, Mareike Hartmann, et al. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[19] Arthur Hosseini et al. Browsergym: A new environment for web agents, 2023.

[20] Daya Guo et al. Repocoder: Repository-level code completion through iterative retrieval and generation. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2471–2484, 2023.

[21] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. Autocoderover: Autonomous program improvement. In *International Symposium on Software Testing and Analysis (ISSTA)*, 2024.

[22] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents, 2024.

[23] Huy Nhat Phan, Tien N. Nguyen, Phong X. Nguyen, and Nghi D. Q. Bui. Hyperagent: Generalist software engineering agents to solve coding tasks at scale, 2024.

[24] Jiayi Pan, Xingyao Wang, Graham Neubig, et al. Training software engineering agents and verifiers with SWE-gym. In *International Conference on Machine Learning (ICML)*, 2025.

[25] Minh V. T. Thai, Tue Le, Dung Nguyen Manh, et al. SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios, 2025.

[26] Qixing Zhou, Jiacheng Zhang, Haiyang Wang, et al. Featurebench: Benchmarking agentic coding for complex feature development, 2026.

[27] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, et al. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents, 2025.

[28] Ziyi Ni, Huacan Wang, Shuo Zhang, et al. Git-taskbench: A benchmark for code agents solving real-world tasks through code repository leveraging. In *AAAI Conference on Artificial Intelligence*, 2026.

[29] Giulio Starace, Oliver Jaffe, Dane Sherburn, et al. Paperbench: Evaluating AI's ability to replicate AI research, 2025.

[30] Qian Huang, Jian Vora, Percy Liang, et al. MLAgentBench: Evaluating language agents on machine learning experimentation. In *International Conference on Machine Learning (ICML)*, pages 20271–20309, 2024.

[31] Cong Wang and Yusheng Zheng. Sandlock: Confining AI agent code with unprivileged linux primitives, 2026.

[32] Georgios Liargkovas, Di Jin, Tianyu (Ezri) Zhu, Dan Liu, A. Bolun Thompson, Anirudh Narsipur, Seong-Heon Jung, Siddhartha Prasad, Diomidis Spinellis, Michael Greenberg, Konstantinos Kallas, and Nikos Vasilakis. hS: Speculative script reordering at subprocess granularity. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pages 665–681. USENIX Association, 2026.

[33] Tianyuan Wu, Chaokun Chang, Lunxi Cao, Wei Gao, and Wei Wang. Crab: A semantics-aware checkpoint/restore runtime for agent sandboxes, 2026.

[34] The strace developers. strace: Linux system call tracer, 2026.

[35] The Linux Kernel Developers. Overlay filesystem. Linux kernel documentation, 2026.

[36] Alexandru Agache, Marc Brooker, Alexandra Iordache, et al. Firecracker: Lightweight virtualization for serverless applications. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 419–434, 2020.

[37] Ethan G. Young, Pengfei Zhu, Tyler Caraza-Harter, et al. The true cost of containing: A gvisor case study. In *USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*, 2019.

[38] Thomas Pasquier, Xueyuan Han, Thomas Moyer, Adam Bates, Olivier Hermant, David Eyers, Jean Bacon, and Margo Seltzer. Runtime analysis of whole-system provenance. In *ACM Conference on Computer and Communications Security (CCS)*, 2018.

[39] Bolaji Gbadamosi, Luigi Leonardi, Tobias Pulls, Toke Hoiland-Jorgensen, Simone Ferlin-Reiter, Simo Sorce, and Anna Brunstrom. The ebpf runtime in the linux kernel, 2024.

[40] Wanqi Yang, Pengfei Chen, Kai Liu, and Huxing Zhang. Nahida: In-band distributed tracing with ebpf, 2023.